

MetabolicCity AI: Tech Stack & System Constraints

Next.js (Frontend) + NestJS (Backend) Monolithic Architecture Specification

Stack Lock Policy: This TRD freezes the architecture for MetabolicCity AI. AI prompt engines and developers must strictly adhere to Next.js 14+ and NestJS 10. No alternative libraries, ORMs, or databases may be substituted during implementation.

1. Technical Stack Specification (LOCKED)

SECTION	WHAT TO SPECIFY	METABOLICCITY AI IMPLEMENTATION
Frontend	Framework + version, styling, state, routing	Next.js 14 (App Router) , React 18, Tailwind CSS, Zustand (global state), TanStack Query v5, Leaflet / Mapbox GL JS, h3-js
Backend	Language, framework, runtime version	TypeScript / Node.js 20 LTS, NestJS 10 (Modular Monolith), REST API (`@nestjs/common`), WebSockets (`@nestjs/platform-socket.io`)
Database	Engine + version, ORM, migrations	MongoDB Atlas v7.0 (M0 Free Tier) , Mongoose 8.x ODM, Change Streams natively enabled
Auth	Method, provider, session model	JWT (NestJS `@nestjs/jwt` + Passport) , 24h Bearer Access Token, Stateless, Roles: MUNICIPAL_OPERATOR, FIELD_CREW, MUNICIPAL_ADMIN
Third-party APIs	Every external service + purpose + limit	• Uber H3 Engine: Spatial hex grid indexing (NPM h3-js) • OpenWeatherMap API: Weather/Rainfall telemetry (1,000 calls/day free) • Twilio / Fast2SMS: Field Crew SMS dispatch gateway
Hosting	Where each piece is deployed	• Frontend: Vercel (Hobby Free Tier) • Backend: Render / Railway (Free Web Service Tier) • Database: MongoDB Atlas (M0 Shared 512MB Cluster)
Constraints	Hard limits AI must respect	100% Free Tiers only; Zero paid APIs; Monolith architecture (No microservices); Load time under 2.5s; Zero Silent Failures
Conventions	Naming, folder structure, error format	camelCase JS/TS, snake_case DB fields, PascalCase NestJS modules, /api/v1/* routes, Standardized Error JSON

SECTION	WHAT TO SPECIFY	METABOLICITY AI IMPLEMENTATION
		envelope

2. System Constraints (Negative Space & Rules)

Constraints prevent architectural bloat and ensure the application remains deployable within free-tier infrastructure bounds.

- BUDGET Zero Paid Infrastructure:** All services must run on MongoDB Atlas M0, Vercel Free, and Render Free Tiers. No card-required services permitted.
- SKILL & COMPLEXITY Modular NestJS Monolith:** Do NOT split backend into microservices, gRPC services, or Docker/Kubernetes clusters. NestJS modules handle separation of concerns within a single process.
- PERFORMANCE Detection Latency Target:** Incoming telemetry updates must trigger Composite Risk Score recalculation (\$R_c\$) and emit WebSocket alerts in under 3 minutes. Frontend map grid rendering must not exceed 100ms frame latency.
- ANTI-PATTERNS Explicit Prohibitions:**
 - DO NOT use Redux, GraphQL, or Prisma for MongoDB (use Zustand and Mongoose 8.x).
 - DO NOT build consumer/citizen login portals (Citizens submit via unauthenticated public endpoints or hotlines).
 - DO NOT allow silent exception swallowing (``catch(e) {}``). All errors must return explicit JSON responses.

3. System Architecture & Folder Layout

```
+-----+
|                                     NEXT.JS 14 FRONTEND (Vercel)                                     |
| /app                               |
|   ├── (auth)/login/page.tsx        ---> Role Access Gateway (Operator / Field Crew / Admin) |
|   ├── (dashboard)/operator/page.tsx ---> Spatial Risk Grid Map (Leaflet/Mapbox + H3-js)      |
|   └── (dashboard)/admin/page.tsx    ---> Seasonal Risk Weight Tuning ($W_m, W_c, W_v$)      |
+-----+
|                                     | REST API & WebSockets                                     |
|                                     v                                     |
+-----+
|                                     NESTJS 10 BACKEND (Render / Railway)                                     |
| /src                               |
|   ├── modules/auth                 ---> JWT Auth, Passport Strategy, Role Guards           |
|   ├── modules/telemetry             ---> Ingestion (GTFS, OpenWeather, Unauth Citizen 311 Reports)|
|   ├── modules/risk-engine           ---> Uber H3 Indexing, $R_c$ Computation Pipeline       |
|   ├── modules/incidents             ---> Change Stream Watcher & Dispatch WebSocket Gateway  |
|   └── common/filters                ---> Global Zero-Silent-Failure Exception Filter       |
+-----+
|                                     | Mongoose ODM                                     |
|                                     v                                     |
+-----+
|                                     MONGODB ATLAS M0 CLUSTER (Free Tier)                                     |
| Collections: `spatial_cells`, `telemetry_events`, `incidents`, `system_config`             |
+-----+
```

```

metaboliccity-ai/
├── apps/
│   ├── web/                                # Next.js 14 App Router (Frontend)
│   │   ├── src/
│   │   │   ├── app/                        # Next.js App Routes
│   │   │   ├── components/                # MapBox H3 Grid, Operator Console, Alert Banners
│   │   │   ├── store/                     # Zustand Global State
│   │   │   └── lib/                       # API Client & Axios setup
│   │   ├── tailwind.config.js
│   │   └── package.json
│   └── api/                               # NestJS 10 Framework (Backend API)
│       ├── src/
│       │   ├── modules/
│       │   │   ├── auth/                  # Controllers, Services, JWT Guards
│       │   │   ├── telemetry/             # Ingestion API Endpoints
│       │   │   ├── risk-engine/           # H3 Math Engine ( $R_c = W_m \cdot S_m + W_c \cdot S_c + W_v \cdot S_v$ )
│       │   │   ├── incidents/             # Dispatch & Workflow State
│       │   │   └── websocket/             # Socket.io Real-time Gateway
│       │   ├── common/                   # Interceptors & Exception Filters
│       │   ├── schemas/                  # Mongoose MongoDB Schemas
│       │   ├── main.ts
│       │   └── package.json
│       └── package.json

```

4. API Conventions & Error Standards

All REST endpoints follow strict JSON formatting. Under the **Zero Silent Failure** rule, all failures explicitly return standard error payloads.

Standard Endpoint Rules

- Base Path Prefix: /api/v1
- Authentication: Authorization: Bearer <JWT_TOKEN>

Standard Success Response Format

```

{
  "success": true,
  "statusCode": 200,
  "data": {
    "h3_index": "8826856235ffffff",
    "composite_score": 7.415,
    "status": "CRITICAL"
  },
  "timestamp": "2026-08-30T19:30:00.000Z"
}

```

Standard Exception Response Format (Zero Silent Failure)

```

{
  "success": false,
  "statusCode": 400,
  "error": {
    "code": "ERR_INVALID_GEO_COORDINATES",

```

```
"message": "Latitude [95.12] exceeds valid geographical bounds (-90 to +90).",
"path": "/api/v1/telemetry/report",
"details": [
  "Field 'latitude' must be between -90 and 90."
]
},
"timestamp": "2026-08-30T19:30:05.000Z"
}
```

5. Copy-Paste TRD Master File (TRD.md)

Use the raw markdown block below to lock your repository configuration directly in code.

```
# TRD – Technical Requirement Document [MetabolicCity AI]

## Stack (LOCKED – do not substitute)
Frontend : Next.js 14 (App Router), Tailwind CSS, Zustand, TanStack Query v5, Leaflet/Mapbox GL JS, h3-js
Backend  : Node.js 20 LTS, NestJS 10 (TypeScript Monolith), `@nestjs/platform-socket.io`
Database : MongoDB Atlas M0 (Free Tier v7.0+), Mongoose 8.x ODM
Auth     : JWT Stateless Bearer Token (`@nestjs/jwt`), NestJS Passport Guards (Roles: OPERATOR, FIELD_CREW, ADMIN)
Hosting  : Frontend -> Vercel; Backend -> Render / Railway; Database -> MongoDB Atlas

## Third-party services
| Service           | Purpose                               | Free tier limit           |
| -----          | -----                               | -----                   |
| Uber H3           | Geospatial Hexagon Indexing         | Open Source (NPM `h3-js`) |
| OpenWeatherMap    | Real-time rainfall/weather data      | 1,000 calls / day         |
| Fast2SMS/Twilio   | SMS Field Dispatch Alerts           | Trial credits / Mock Mode  |

## Constraints (HARD RULES)
- Budget : 100% Free Tiers only. No paid APIs or credit card-gated services.
- Skill  : NestJS Modular Monolith. Do NOT build microservices or Docker clusters.
- Perf   : Max 3-minute latency from raw telemetry update to dispatcher socket emission.
- Anti   : DO NOT USE: Redux, GraphQL, Prisma, Docker Compose, Citizen Login modules.

## Conventions
- Naming : camelCase JS/TS, snake_case DB fields, PascalCase NestJS Services/Modules
- Folders : Monorepo layout (`/apps/web` for Next.js, `/apps/api` for NestJS)
- API     : Prefix `/api/v1/*`
- Error   : Explicit JSON schema `{ success: false, statusCode, error: { code, message } }`. Zero silent failures
```

6. Architectural Review Report

Specification Audit Item	Verdict	Engineering Notes
Tech Stack Alignment	✓ LOCKED	Next.js 14 App Router + NestJS 10 modular monolith specified.
Database Tier Compatibility	✓ PASSED	MongoDB Mongoose 8.x schema tailored for MongoDB Atlas M0 Free Tier.
Auth Model Verification	✓ PASSED	Strictly 3 roles (Operator, Field Crew, Admin). Citizen login omitted.

SPECIFICATION AUDIT ITEM	VERDICT	ENGINEERING NOTES
Error Observability	PASSED	Zero Silent Failure standard implemented across NestJS filters.